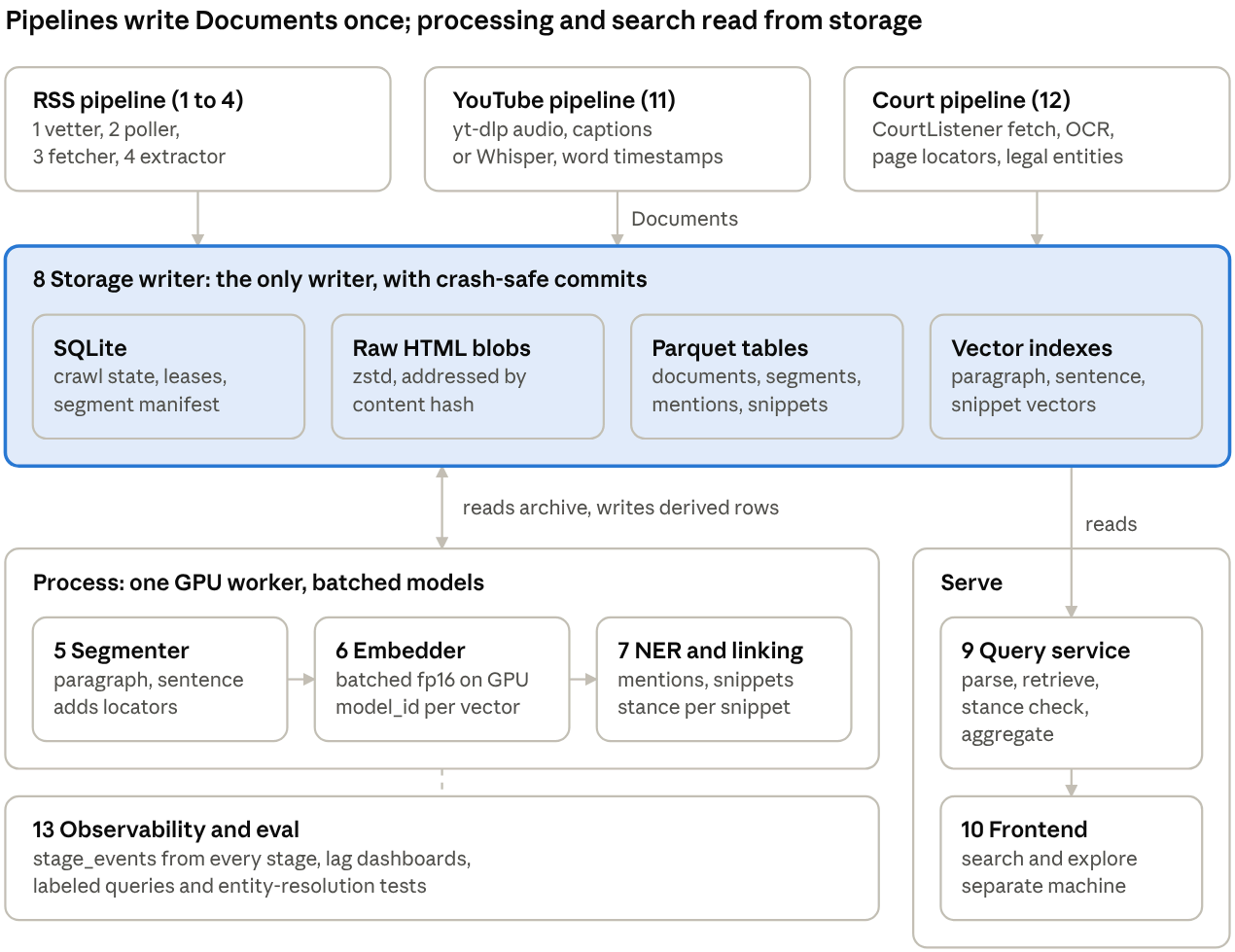
News RAG Search Engine: Project Overview

Oct 15, 2026 · @Dawson

A local-first pipeline ingests RSS, YouTube and court documents into one Document-to-Segment model, then serves entity-filtered, stance-aware semantic search over it.

At a glance



architecture · 13 components in 4 zones

Every pipeline emits the same Document record, the storage writer is the single commit point, and the GPU worker reads the archive and writes derived rows. The query service reads storage, and the frontend runs on a different machine and reaches it only through the query API.

Goals, non-goals and success metric

The system must answer an opinion query (for example "Trump is a good guy for energy") with the articles, and the exact lines, that express that stance, while new content stays searchable within minutes.

Goals

- **Search mode:** a free-text opinion query plus optional entity and date filters returns ranked articles with the matching sentences highlighted.
- **Explore mode:** browse the entity map (articles mentioning two entities, a publisher's coverage, publishers covering an entity, an entity in a date range).
- **Pluggable pipelines:** RSS first, then YouTube (transcripts with timestamps), then CourtListener, all feeding one core.
- **Local-first AI:** every model runs on the backend machine (i7, RTX 2070 with 6GB VRAM).
- **Crash-safe and live:** the backend can lose power without corrupting data, and the frontend can query while ingestion runs.
- **Distributable:** each component sits behind a versioned interface so it can move to another machine.

Non-goals for now

- Multi-user auth, uptime guarantees and UI polish.
- Real-time (sub-minute) discovery of every publisher.
- A hand-built ANN index. Use an existing engine unless this becomes a deliberate learning project.

Success metric: ingest lag

Three lags, each tracked as p50, p95 and p99 per source and per stage: discovery lag (discovered minus published), ingest lag (searchable minus discovered) and end-to-end lag (searchable minus published). Ingest lag is the one the system controls.

Conventions every component follows

These rules are what let components be rebuilt independently, so every drill-down brief should restate them.

- **Document IDs:** `doc_id` is the first 32 hex characters of sha256 of the canonical URL, so retries and re-runs are idempotent.

- **Revisions:** the same `doc_id` with a new `content_hash` is a new `revision`. Different `doc_id`s with the same `content_hash` are duplicate candidates, kept and linked later (`dup_of`), not collapsed.
- **Timestamps:** UTC everywhere. `published_at` is nullable and is never replaced by `ingested_at`; `discovered_at` and `ingested_at` are separate fields.
- **Versioning:** every row carries `schema_version`, and every derived row carries `pipeline_version` plus the `model_id` or `chunker_version` that produced it. Breaking contract changes go in a new `contracts/v2/` directory.
- **Locators:** every Segment has a locator that points back into its source: character offsets for text, a time range for video, a page and offset for court PDFs.
- **Derived data is rebuildable:** raw HTML and extracted text are the source of truth. Embeddings, mentions and entity links can always be recomputed.
- **Links are scored edges:** entity resolution writes confidence-scored edges, never hard merges.
- **Stage events:** every stage writes a `stage_events` row (enqueued, started, finished, worker) so lag is measurable from day one.
- **Crash safety:** write to a temporary file, fsync, atomic rename, then update the manifest in one SQLite transaction. A file not in the manifest does not exist.

Components

The system splits into 13 components, and each row below is one drill-down target. "Scales by" names the parallelism that actually helps that stage.

#	Component	Job	Reads → writes	Scales by	Status
1	Feed vetter	Scores each feed for paywalls, full text, date quality, robots	Feed list → FeedHealthReport	Run on demand	Contract drafted
2	Feed poller	Conditional GETs with adaptive intervals	Feeds → discovered URLs	One async process	Contract drafted
3	Article fetcher	Throttled async fetch, stores raw HTML	URLs → raw blobs	Shard by domain	Contract drafted, design doc next
4	Extractor	HTML to text, publication date, language, content kind	Raw blob → Document	Process pool	Contract drafted

#	Component	Job	Reads → writes	Scales by	Status
5	Segmenter	Splits Documents into paragraphs and sentences with locators	Document → Segment	Process pool	Not started
6	Embedder	Batched GPU embedding with `model_id` on every vector	Segment → vectors	GPU batching	Not started
7	NER and entity linker	Mentions, snippets, entity-targeted sentiment, shortname resolution	Segment → Mention, Snippet, Entity edges	GPU batching	Not started
8	Storage writer	Segment commit, manifest, compaction, recovery	Enriched rows → committed segments	One writer per table	Protocol drafted, engine undecided
9	Query service	Parse query, retrieve, filter, stance rerank, aggregate	Query → ranked results	Replicas	Not started
10	Frontend	Query preprocessing, search mode, explore mode	User → query API	Separate machine	Not started
11	YouTube pipeline	yt-dlp audio, captions or Whisper, word timestamps, diarization	Channel feed → Document	Per-video	Not started
12	Court pipeline	CourtListener fetch, OCR, legal entities, page locators	Docket → Document	Per-document	Not started
13	Observability and eval	Lag dashboards, labeled query set, entity-resolution test set	stage_events, labels → reports	Run on demand	Not started

Components 1 to 4 plus the storage writer form Phase 1. Components 5 to 7 are Phase 2, and 9 and 10 are Phase 3 (see Build order).

Core data model

Every pipeline emits a Document, and everything downstream hangs off Document and Segment, so these two contracts matter most. The Phase 1 contracts (Document, crawl state, feed health) are specified in `contracts/v1/`.

Record	Key fields	Notes
Document	`doc_id`, `revision`, `source_id`, `pipeline`, `canonical_url`, `title`, `extracted_text`, `content_kind`, `published_at` + source, `fetched_at`, `ingested_at`, `content_hash`, `raw_blob_hash`, `extractor_version`	One row per revision. `content_kind` is full_text, summary_only or suspected_paywall.
Segment	`segment_id`, `doc_id`, `level` (paragraph or sentence), `parent_id`, `position`, `text`, `locator`, `chunker_version`	Sentences point to their paragraph. For video, a paragraph is a run of transcript segments grouped by time or speaker turn.
Embedding	`segment_id` or `snippet_id`, `model_id`, `dims`, `vector`, `embedded_at`	Article-level vector is derived (mean of paragraph vectors), not separately embedded.
Mention	`mention_id`, `doc_id`, `segment_id`, `surface`, `char_span`, `ner_label`, `in_title`, `ner_model_version`	`in_title` separates "Article is about Entity" from "Article mentions Entity".
Entity	`entity_id`, `canonical_name`, `type`, `aliases`, `kb_id` (nullable, for example Wikidata)	Types: person, place, organization, plus a closed set of others.
MentionLink	`mention_id`, `entity_id`, `confidence`, `method`, `features`	Scored edge. Short names like "Paris" can link to several candidates. Unresolved mentions stay as their own nodes.
Snippet	`snippet_id`, `doc_id`, `entity_id`, `text`, `locator`, `stance_score` (nullable), `stance_model_version`	N words either side of a mention. Join point between the entity map and the vector index.

Record	Key fields	Notes
Relations	`source_id` published `doc_id`, `dup_of`, article sentiment	Only relation types with high measured accuracy go in. Entity-targeted sentiment lives on Snippet, not Document.
StageEvent	`doc_id`, `revision`, `stage`, `enqueued_at`, `started_at`, `finished_at`, `worker_id`	Drives all lag metrics.

Crawl state (feeds, polls, URL frontier with leases, segment manifest, per-domain limits) is mutable and lives in SQLite, not in the table above.

Storage and indexing

Parquet is a good file format for this data but not a database, so storage is split by access pattern. Parquet gives roughly 3 to 5 times compression on text and fast DuckDB scans, but it is immutable and has no transactions, indexes or crash safety.

Data	Store	Why
Crawl state, leases, segment manifest	SQLite (WAL, synchronous FULL); Postgres if the fetchers span machines	Mutable and transactional
Raw HTML	Content-addressed zstd blobs, `raw/ab/cd/<hash>.html.zst`	Write once, rarely read, about 20 to 60KB each
Documents, Segments, Mentions, Snippets	Parquet segments queried with DuckDB, or LanceDB / Delta Lake	Append-mostly and analytical
Vector indexes	The ANN engine's own format (Qdrant, LanceDB, FAISS or hnswlib)	Parquet is not an index
stage_events	SQLite or Parquet	Small, queried for dashboards

Rules

- Only the storage writer touches segments. Commit order is blob, then Parquet segment, then one SQLite transaction that records the manifest row and marks the URLs stored.
- On startup, delete temp files, delete or verify orphan segments, and requeue URLs whose lease expired.
- Never share a Parquet or SQLite file over a network mount between machines. Serve queries through an API, or replicate immutable segments.
- Hold a small in-memory "recent" index beside the main ANN index so new articles are searchable before the next index rebuild.
- Filtered vector search (entity plus date range) drives engine choice more than anything else. Options are per-entity partitions, an engine with native filtered search, or oversample then filter.

Pending decision: prototype Parquet-plus-DuckDB against LanceDB on a few thousand articles and compare size, write latency and query time before committing. The contracts are storage-agnostic, so the choice stays swappable.

Query path

Embeddings match topic, not stance, so an opinion query needs a retrieve-then-verify pipeline. "Trump is good for energy" and "Trump is terrible for energy" sit close together in vector space.

1. **Parse the query:** extract the entity (Trump), the aspect (energy) and the polarity (positive), and resolve the entity to an `entity_id`. Time-range parsing happens here too. The frontend may do this step, but it must use the same embedding `model_id` as the backend.
2. **Gate with the entity map:** select articles and snippets that mention the entity, within the date filter.
3. **Retrieve for recall:** search the paragraph index within that gate, taking a generous top-K (200 to 500).
4. **Locate lines:** score the sentences inside the top paragraphs against the query to find the exact lines to highlight.
5. **Judge stance:** run an NLI model (query as hypothesis, entity snippet as premise) to label each candidate as supporting, neutral or contradicting. Optionally cascade the top 30 to 50 to a small local LLM for implicit stance.

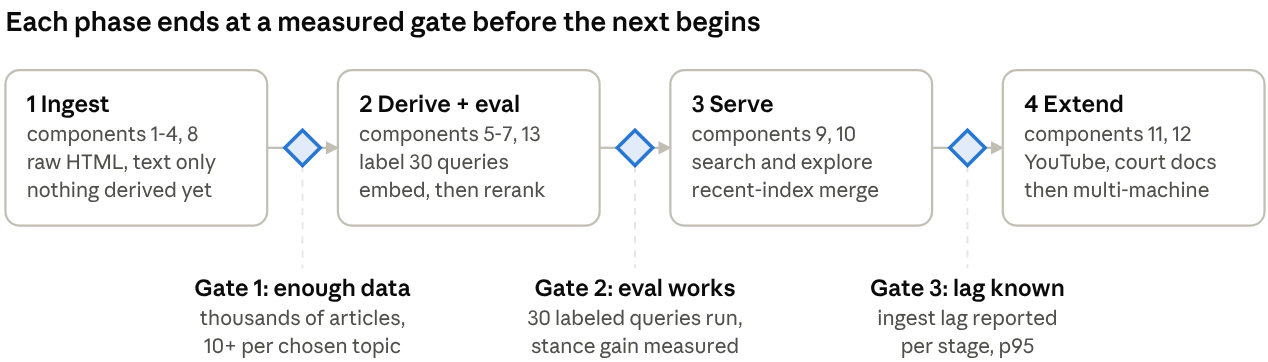
6. **Aggregate:** roll sentence scores up to articles (max, mean, or supporting versus contradicting count), then group by publisher and apply date filters. Show contradicting articles in a separate "opposing views" section.
7. **Fallback:** if entity resolution misses a mention, that snippet never reaches stance judgment. Keep a sentence-level search that does not depend on the entity map.

Why snippets for stance: a bare sentence like "He signed the order Tuesday" has an unresolved subject. A snippet tied to a resolved entity carries its target.

Fast path (later): precompute entity-targeted sentiment per snippet at ingest, so common queries become filtered lookups. It cannot cover stances outside the predefined schema.

Known weak spots: implicit stance in factual sentences, reported versus endorsed opinion ("Critics say Trump is wrong"), sarcasm and hedging. Treat scores as ranking signals, not facts.

Build order



roadmap · 4 phases, 3 gates

Phase 1 stores raw HTML and extracted text only, so chunking, embeddings and NER can be re-run on the archive later. Labeling the 30 queries starts once Gate 1 is met, and new components are added behind the contracts rather than rewriting them.

Open decisions

Ten decisions are still open, and each blocks a specific component. The default column is what to assume if no answer arrives, so work can proceed.

Decision	Blocks	Default if undecided
Learning project (hand-build HNSW and entity linker) or product (use Qdrant, LanceDB, DuckDB)	6, 7, 8	Product: use existing engines
Expected scale: articles per day and years of retention	6, 8	5k articles per day, 2 years
Match topic only, or stance too	9	Stance via NLI rerank
Entity map a core deliverable, or a filter and exploration aid	7, 10	Core deliverable, built after Phase 2 retrieval works
Entity-resolution error tolerance: precision or recall first	7	Precision first, unresolved mentions stay unlinked
One backend machine or a real cluster	3, 8	One backend machine plus one frontend machine
Users: only the author, or others	9, 10	Author only, no auth
How "good" results are judged	13	30 labeled opinion queries with supporting, opposing and off-topic articles
Dedup and revision policy	4, 8	Keep every copy and every revision, link with `dup_of`
Storage engine	8	Decide after the Parquet-versus-LanceDB prototype

Drill-down guide

To design or build one component, give the AI this overview plus a brief in the shape below, one component per session. Each component's design doc should answer every field, so any module can be upgraded behind its interface.

```
# Component: <name and number from the Components table>

## Context
Paste: Goals, Conventions, Core data model, and the rows for this component's
neighbours.
```

```
## Purpose
One sentence: what this component does and why it exists.

## Interface
- Inputs: record types, source (queue, table or API), delivery semantics
- Outputs: record types, destination, ordering and idempotency guarantees
- Contract files: contracts/v1/<file>

## Behaviour
Steps, state machine, retry and backoff rules, lease handling.

## Failure modes
What breaks, how it is detected, how it recovers after a crash or restart.

## Performance
Bound by (I/O, CPU, GPU), batch sizes, queue limits, lag targets, metrics
emitted to stage_events.

## Test plan
Golden fixtures, contract tests, eval set, acceptance criteria.

## Open questions
Decisions from the Open decisions table that affect this component.
```

Order of drill-downs

1. Contracts first: Document, Segment, Mention, Entity, Embedding and the query API, as versioned schemas in `contracts/`.
2. Fetch-and-extract worker design doc (leases, per-domain throttling, backoff, adaptive polling).
3. Golden fixtures: saved feed XML and article HTML, including a paywalled page and a summary-only feed.
4. Storage writer and the Parquet-versus-LanceDB prototype.
5. Segmenter, embedder and the retrieval eval, then NER and entity linking, then the query service.

Working conventions for Claude sessions

- Keep one session per component, with a repository `CLAUDE.md` that holds conventions and the contract locations.

- Build a walking skeleton first (3 feeds, one machine, end to end to a basic search page), then split across machines.
- Use Claude during development to propose queries and silver labels for the eval sets. Verify the labels by hand, since they are the ground truth.
- Local models run in production. Claude is a development tool here, not a runtime dependency.